

# Argo CD : Guide Complet

## GitOps pour Kubernetes - Explication détaillée, commandes et utilité dans un projet

Version 2026 - Par Grok (basé sur Argo CD v3.x stable)

## Qu'est-ce que Argo CD ?

Argo CD est un **outil open-source de Continuous Delivery (CD) déclaratif basé sur GitOps** spécialement conçu pour Kubernetes.

Il surveille en permanence tes dépôts Git (source de vérité) et **synchronise automatiquement** l'état de tes clusters Kubernetes pour qu'il corresponde exactement à ce qui est décrit dans Git.

### Phrase clé à retenir :

« Git est la source de vérité unique. Argo CD fait le reste. »

## Pourquoi Argo CD ? (GitOps)

- Plus de « ça marche en dev mais pas en prod »
- Déploiements reproductibles, auditable et rollback en 1 clic
- Tout est versionné dans Git (manifests, Helm, Kustomize)
- Pull-based : Argo CD tire les changements (pas de push de credentials Kubernetes)
- Interface web magnifique + CLI puissante
- Support multi-clusters, multi-tenant, haute disponibilité

### Comparaison rapide avec Flux CD :

Argo CD = meilleure interface graphique + plus intuitif pour les équipes

Flux CD = plus léger, plus CLI-first

## Concepts clés (très simples)

| Concept            | Définition simple                                                | Exemple concret                         |
|--------------------|------------------------------------------------------------------|-----------------------------------------|
| <b>Application</b> | Une « app » que tu veux déployer (défini dans un CRD Kubernetes) | Ton site web + base de données          |
| <b>AppProject</b>  | Groupe d'applications avec restrictions (sécurité)               | Projet « production » ou « équipe dev » |
| <b>Source</b>      | Où est le code (Git + chemin + branche/tag)                      | repo GitHub + /k8s/ + main              |
| <b>Destination</b> | Où déployer (cluster + namespace)                                | cluster-prod + namespace monapp         |

| Concept                         | Définition simple                                        | Exemple concret                  |
|---------------------------------|----------------------------------------------------------|----------------------------------|
| <b>Sync</b>                     | Action de mise à jour (manuel ou automatique)            | Argo CD applique les changements |
| <b>Health &amp; Sync Status</b> | Vert = tout est OK / Rouge = problème                    | Icônes claires dans l'UI         |
| <b>Prune &amp; Self-Heal</b>    | Supprime les ressources en trop / Répare automatiquement | Sécurité avancée                 |

## Installation rapide (2026)

### 1. Sur un cluster Kubernetes (recommandé)

```
# Créer le namespace
kubectl create namespace argocd

# Installation rapide (non-HA pour test)
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/mã

# Pour production (HA + Helm - recommandé)
helm repo add argo https://argoproj.github.io/argo-helm
helm install argocd argo/argo-cd -n argocd --create-namespace
```

### \*\*2. Installer la CLI argocd macOS :

```
brew install argocd
```

Linux :

```
curl -sSL -o /usr/local/bin/argocd https://github.com/argoproj/argo-cd/releases/latest/
```

Windows : Télécharge depuis GitHub Releases

### \*\*3. Accéder à l'UI

```
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

Ouvre <https://localhost:8080> Utilisateur par défaut : admin Mot de passe :

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}'
```

## Commandes CLI argocd les plus utiles

```

# Connexion
argocd login localhost:8080 --username admin --password xxx

# Lister toutes les applications
argocd app list

# Créer une application
argocd app create monapp \
  --repo https://github.com/monorg/monrepo.git \
  --path k8s/ \
  --dest-server https://kubernetes.default.svc \
  --dest-namespace mon-namespace

# Synchroniser (déployer)
argocd app sync monapp

# Voir le statut en temps réel
argocd app get monapp --watch

# Forcer un refresh Git
argocd app refresh monapp

# Supprimer une application (sans supprimer les ressources)
argocd app delete monapp --cascade=false

```

## Exemple complet d'Application (fichier YAML dans Git)

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: mon-app-web
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/monorg/mon-app.git
    targetRevision: main
    path: manifests/           # ou helm/ ou kustomize/
    helm:
      valueFiles:
        - values-prod.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated:
      prune: true           # supprime les ressources en trop
      selfHeal: true        # répare automatiquement
    syncOptions:
      - CreateNamespace=true

```

## Comment ça marche avec Docker ?

- Tu builds ton image Docker → `docker build -t monimage:v1.2` .
- Tu la pushes vers Docker Hub / GitHub / ECR
- Dans ton repo Git, tu mets à jour le Deployment YAML ou le Helm values.yaml avec `image: monimage:v1.2`
- Tu commit & push sur main
- Argo CD détecte le changement en < 3 minutes et déploie automatiquement sur Kubernetes !

## Utilité de Argo CD dans un projet (2026)

- Zéro erreur de déploiement - tout est dans Git
- Rollback en 1 clic (git revert + sync)
- Environnements multiples (dev/staging/prod) avec un seul repo ou plusieurs
- Équipes en sécurité (RBAC par projet)
- Observabilité incroyable (UI montre les diffs en temps réel)
- Intégration CI/CD parfaite (GitHub Actions / GitLab CI push → Argo CD déploie)
- Multi-clusters (gère 10, 50, 100 clusters depuis une seule instance)
- App of Apps (Argo CD déploie d'autres Argo CD Applications - pattern avancé)

## Exemple concret :

Une startup avec 5 microservices (frontend React + backend Node + Redis + Postgres + Nginx). → Tout dans un repo Git → git push = déploiement complet en 2 minutes sur 3 environnements → Nouveau dev : clone le repo → tout marche

## Conclusion

- Argo CD = le standard GitOps de Kubernetes en 2026.
- Avec Docker (pour construire les images) + Kubernetes (pour orchestrer) + Argo CD (pour déployer), tu as la stack moderne complète, fiable et professionnelle.
- Commande à retenir pour commencer :

```
argocd app sync monapp
```